

GOVERNMENT DEGREE COLLEGE BEERWAH

PYTHON PROGRAMMING

Introduction to Python

6th Semester (BCA)

Unit 2

Functions

Functions are reusable blocks of code that perform a specific task. They help in organizing code, improving readability, and reducing redundancy.

Defining a Function

involves creating a reusable block of code that performs a specific task. Functions are defined using the **def** keyword, followed by the function name, a pair of parentheses `()`, and a colon `:`. The block of code inside the function is indented and executed when the function is called.

Syntax

```
def function_name(parameters):  
    # Function body  
    # Code to execute
```

def Keyword used to define a function.

function_name Name of the function (follows the same rules as variable names).

parameters Optional inputs to the function (enclosed in parentheses).

Function body The indented block of code that defines what the function does.

Return Value Functions can return a value using the **return** statement.

If no **return** statement is provided, the function returns **None**

Calling a Function

means executing the code defined within the function. Once a function is defined, you can call it by using its name followed by parentheses `()`. If the function accepts parameters, you pass the required arguments inside the parentheses.

Syntax

```
function_name(arguments)
```

function_name: Name of the function to call.

arguments Values passed to the function (if it accepts parameters).

If the function has parameters, provide the required arguments in the correct order.

Arguments can be positional, keyword, or default.

return value If the function returns a value, you can store it in a variable or use it directly.

Parameters and Arguments .

Parameters and **Arguments** are closely related concepts used in functions. They allow you to pass data into a function, making it more flexible and reusable.

Parameters:

- ◇ **Definition:** Parameters are variables listed inside the parentheses in the function definition.
- ◇ **Purpose:** They act as placeholders for the values (arguments) that will be passed to the function when it is called.
- ◇ **Scope:** Parameters are local to the function and cannot be accessed outside of it.

Syntax

```
def function_name(parameter1, parameter2, ...):  
    # Function body
```

Arguments:

- ◇ **Definition:** Arguments are the actual values passed to the function when it is called.
- ◇ **Purpose:** They provide the data that the function will use.
- ◇ **Scope:** passed from outside the function, when function is called.

Syntax

```
function_name(argument1, argument2, ...)
```


Parameters and Arguments ..

Types of Arguments

◆ Formal and Actual Arguments

- ◆ **Formal Arguments:** These are the variables listed within a function definition that are expected to receive values when the function is called.
- ◆ **Actual Arguments:** These are the concrete values that are actually supplied to a function when it is invoked.

◆ Positional Arguments

- ◆ When calling a function, positional arguments must be provided in the exact order as defined in the function definition.

◆ Keyword Arguments

- ◆ Arguments are passed with the parameter name, allowing you to change the order of arguments.

◆ Default Arguments

- ◆ A function can have arguments with pre-defined default values, so if a caller doesn't explicitly provide a value for that argument, the default is used.

◆ Variable Length Arguments

- ◆ Functions can accept any number of arguments using **args* and ***kwargs*.

Return Values and return Statements

Return values and **Return statements** are used to send data back from a function to the caller. A function can perform computations or operations and then return a result using the return statement. If no return statement is provided, the function implicitly returns None.

Return Values:

- ✧ A **return value** is the value that a function sends back to the caller.
- ✧ It can be any data type, such as integers, strings, lists, dictionaries, or even other functions.

Return Statement:

- ✧ The **return statement** is used to exit a function and return a value.
- ✧ Once a return statement is executed, the function terminates immediately, and no further code in the function is executed.

Syntax

```
def function_name(parameters):  
    # Function body  
    return value
```

variable = function_name(arguments)

Key Points:

- ✧ **Single Return Value:**
 - ✧ A function can return a single value.
- ✧ **Multiple Return Values:**
 - ✧ A function can return multiple values as a tuple.
- ✧ **No Return Statement:**
 - ✧ If a function does not have a return statement, it implicitly returns None.
- ✧ **Early Exit:**
 - ✧ The return statement can be used to exit a function early.

The None Value

None is a special constant that represents the absence of a value or a null value. It is often used to indicate that a variable or function does not have a meaningful value to return. None is an object of the *NoneType* class and is the sole instance of that class.

Key Characteristics of None

Singleton Object:

- ◊ There is only one none Object in Python, and it is immutable.
- ◊ You can check if a variable is None using the is operator.

Default Return Value:

- ◊ If a function does not explicitly return a value, it implicitly returns None.

Placeholder:

- ◊ None is often used as a placeholder to indicate that a variable has not been assigned a value yet.

Comparison:

- ◊ None is falsy in Boolean contexts, meaning it evaluates to False.

Scope

In Python, *scope* refers to the region of a program where a variable or name is accessible. Understanding scope is crucial for managing variables and avoiding conflicts in your code.

Local Scope

- ◇ Variables defined inside a function have *local scope*.
- ◇ They are accessible only within that function and not outside of it.
- ◇ Local variables are created when the function is called and destroyed when the function exits.

Global Scope

- ◇ Variables defined outside all functions have *global scope*.
- ◇ They are accessible throughout the entire program, including inside functions.
- ◇ However, to modify a global variable inside a function, you must use the *global* statement.

The global Statement

- ◇ The *global* keyword is used to declare that a variable inside a function refers to a global variable.
- ◇ Without *global* assigning a value to a variable inside a function creates a new local variable.

The List Data Type

List in Python is an ordered, mutable (changeable), and heterogeneous (can store different data types) collection of items. Lists are one of the most commonly used data structures in Python.

Creating Lists

Lists are created by enclosing elements in square brackets `[]` separated by commas.

List of strings

```
fruits = ["apple", "banana", "cherry"]
```

List of integers

```
numbers = [1, 2, 3, 4, 5]
```

Mixed data types

```
mixed = [1, "apple", 3.14, True]
```


Basic List Operations .

Accessing Elements:

- ◇ Use indexing (e.g., *list[0]*).

Modifying Elements:

- ◇ Assign a new value to an index (e.g., *list[1] = "new"*).

Adding Elements:

- ◇ Use *append()*, *extend()*, or *insert()*.

Removing Elements:

- ◇ Use *remove()*, *pop()*, or *clear()*.

Concatenation:

- ◇ Use *+* to combine lists.

Repetition:

- ◇ Use *** to repeat a list.

Membership Testing:

- ◇ Use *in* to check if an element exists.

Length:

- ◇ Use *len()* to get the number of elements.

Sorting:

- ◇ Use *sort()* or *sorted()*.

Reversing:

- ◇ Use *reverse()* or *slicing*.

Basic List Operations ..

Accessing Elements:

You can access elements in a list using their **index**. Python uses zero-based indexing, meaning the first element is at index 0.

```
fruits = ["apple", "banana", "cherry"]  
print(fruits[0]) # Output: apple  
print(fruits[-1]) # Output: cherry (negative index)
```

Modifying Elements:

Lists are **mutable**, meaning you can change their elements after creation.

```
fruits = ["apple", "banana", "cherry"]  
fruits[1] = "blueberry" # Modify the second element  
print(fruits) # Output: ['apple', 'blueberry', 'cherry']
```

Basic List Operations ...

Adding Elements:

You can add elements to a list using methods like *append()* | *extend()* | *insert()*.

append(): Adds an element to the end of the list.

```
fruits = ["apple", "banana"]  
fruits.append("cherry")  
print(fruits) # Output: ['apple', 'banana', 'cherry']
```

extend(): Adds all elements of an iterable (e.g., list, tuple) to the end of the list.

```
fruits = ["apple", "banana"]  
fruits.extend(["cherry", "date"])  
print(fruits) # Output: ['apple', 'banana', 'cherry', 'date']
```

Insert(): Inserts an element at a specific index.

```
fruits = ["apple", "banana"]  
fruits.insert(1, "orange") # Insert at index 1  
print(fruits) # Output: ['apple', 'orange', 'banana']
```

Basic List Operations

Removing Elements:

You can remove elements from a list using methods like *remove()* | *pop()* | *clear()*

***remove()*:** Removes the first occurrence of a specific value.

```
fruits = ["apple", "banana", "cherry"]  
fruits.remove("banana")  
print(fruits) # Output: ['apple', 'cherry']
```

***pop()*:** Removes and returns the element at a specific index (default is the last element).

```
fruits = ["apple", "banana", "cherry"]  
removed_item = fruits.pop(1) # Remove element at index 1  
print(fruits) # Output: ['apple', 'cherry']  
print(removed_item) # Output: banana
```

***clear()*:** Removes all elements from the list.

```
fruits = ["apple", "banana", "cherry"]  
fruits.clear()print(fruits) # Output: []
```


Basic List Operations

Concatenation:

You can combine two lists using the + operator.

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]
combined = list1 + list2
print(combined) # Output: [1, 2, 3, 4, 5, 6]
```

Repetition:

You can repeat a list using the * operator.

```
list1 = [1, 2, 3]
repeated = list1 * 3
print(repeated) # Output: [1, 2, 3, 1, 2, 3, 1, 2, 3]
```

Membership Testing:

You can check if an element exists in a list using the in keyword.

```
fruits = ["apple", "banana", "cherry"]
print("apple" in fruits) # Output: True
print("mango" in fruits) # Output: False
```

Basic List Operations

Length of a List:

You can find the number of elements in a list using the `len()` function.

```
fruits = ["apple", "banana", "cherry"]  
print(len(fruits)) # Output: 3
```

Sorting a List

You can sort a list in ascending or descending order using the **`sort()`** method or the **`sorted()`** function.

`sort()`: Sorts the list in-place (modifies the original list).

```
numbers = [3, 1, 4, 1, 5, 9]  
numbers.sort()  
print(numbers) # Output: [1, 1, 3, 4, 5, 9]
```

`sorted()`: Returns a new sorted list (does not modify the original list).

```
numbers = [3, 1, 4, 1, 5, 9]  
sorted_numbers = sorted(numbers, reverse=True)  
print(sorted_numbers) # Output: [9, 5, 4, 3, 1, 1]
```

Basic List Operations

Reversing a List:

You can reverse the order of elements in a list using the **reverse()** method or slicing.

reverse(): Reverses the list in place.

```
fruits = ["apple", "banana", "cherry"]  
fruits.reverse()  
print(fruits) # Output: ['cherry', 'banana', 'apple']
```

Copying a List:

You can create a copy of a list using the **copy()** method or slicing.

copy(): Returns a shallow copy of the list.

```
fruits = ["apple", "banana", "cherry"]  
fruits_copy = fruits.copy()  
print(fruits) # Output: ['apple', 'banana', 'cherry']  
print(fruits_copy) # Output: ['apple', 'banana', 'cherry']
```

Indexing and Slicing in Lists .

Indexing and **Slicing** are powerful techniques used to access specific elements or sublists within a list. They allow you to retrieve, modify, or extract portions of a list based on their positions (indices).

Indexing:

- ◇ Indexing refers to accessing individual elements in a list using their **index**. Python uses **zero-based indexing**, meaning the first element is at index 0 the second at index 1, and so on. Negative indices can also be used to access elements from the end of the list.

Syntax

list[index]

```
fruits = ["apple", "banana", "cherry", "date"]
```

```
# Accessing elements using positive indices
```

```
print(fruits[0]) # Output: apple
```

```
print(fruits[2]) # Output: cherry
```

```
# Accessing elements using negative indices
```

```
print(fruits[-1]) # Output: date (last element)
```

```
print(fruits[-2]) # Output: cherry (second last element)
```


Indexing and Slicing in Lists ..

Slicing:

◇ Slicing refers to accessing a sub-list (a portion of the list) using a range of indices.

Syntax

list[start:stop:step]

start: The starting index (inclusive).

stop: The ending index (exclusive).

step: The interval between elements (optional, default is 1).

Examples

```
fruits = ["apple", "banana", "cherry", "date", "elderberry"]
```

Basic slicing

```
print(fruits[1:4]) # Output: ['banana', 'cherry', 'date'] (indices 1 to 3)
```

Omitting start or stop.

```
print(fruits[:3]) # Output: ['apple', 'banana', 'cherry'] (from start to index 2)
```

```
print(fruits[2:]) # Output: ['cherry', 'date', 'elderberry'] (from index 2 to end)
```

Using step

```
print(fruits[::2]) # Output: ['apple', 'cherry', 'elderberry'] (every 2nd element)
```

Negative indices in slicing

```
print(fruits[-3:-1]) # Output: ['cherry', 'date'] (indices -3 to -2)
```

Indexing and Slicing in Lists ...

Common Use Cases for Slicing:

Extracting a Sublist:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
sublist = numbers[2:6] # Extract elements from index 2 to 5
print(sublist) # Output: [3, 4, 5, 6]
```

Reversing a List:

```
fruits = ["apple", "banana", "cherry"]
reversed_fruits = fruits[::-1] # Reverse the list
print(reversed_fruits) # Output: ['cherry', 'banana', 'apple']
```

Copying a List:

```
original = [1, 2, 3]
copy = original[:] # Create a shallow copy
copy.append(4)
print(original) # Output: [1, 2, 3]
print(copy) # Output: [1, 2, 3, 4]
```

Skipping Elements:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
skip_every_second = numbers[::2] # Skip every second element
print(skip_every_second) # Output: [1, 3, 5, 7, 9]
```

Built-In Functions Used on Lists

len(): Get the number of elements in a list.

max(): Get the maximum value in a list.

min(): Get the minimum value in a list.

sum(): Get the sum of all elements in a list.

sorted(): Return a sorted list.

any(): Check if any element is True.

all(): Check if all elements are True.

enumerate(): Get (index, value) pairs.

map(): Apply a function to all elements.

filter(): Filter elements based on a condition.

zip(): Combine multiple lists into tuples.

reversed(): Reverse a list.

list(): Convert an iterable to a list.

List Methods

append(): Add an element to the end of the list.

extend(): Add multiple elements to the end of the list.

insert(): Insert an element at a specific index.

remove(): Remove the first occurrence of a specific value.

pop(): Remove and return an element at a specific index.

clear(): Remove all elements from the list.

index(): Find the index of a specific value.

count(): Count occurrences of a specific value.

sort(): Sort the list in ascending or descending order.

reverse(): Reverse the order of elements in the list.

copy(): Create a shallow copy of the list.

What is a Comprehension?

Definition

A comprehension is a syntactic construct that allows sequences (lists, sets, dictionaries) to be created using a single line of code.

Advantages of Comprehensions

- More concise code
- Better readability
- Can improve performance

Caution:

- Avoid using them for complex logic to keep readability.

Use comprehensions when:

- You need to create a list, set, or dictionary in a concise way.
- The logic is simple and doesn't affect readability.

List Comprehension

Syntax:

[expression for item in iterable if condition]

Example: 1

numbers = [x for x in range(10)]

Example: 2

*squares = [x**2 for x in range(10)]*
print(squares)

List Comprehension with Condition

Example with Condition:

#Program for finding squares of even numbers upto 10

```
even_squares = [x**2 for x in range(10) if x % 2 == 0]  
print(even_squares)
```

List Comprehension with For Loop

Syntax:

[expression for item in iterable]

Example:

```
numbers = [x for x in range(5)]  
print(numbers)
```

The list comprehension iterates over range(5) and stores each number in list

For Loop vs List Comprehension

Traditional for loop

```
numbers = []  
for x in range(5):  
    numbers.append(x)  
print(numbers)
```

List comprehension

```
numbers = [x for x in range(5)]  
print(numbers)
```

List comprehension provides more concise and Pythonic alternative to for loop

List Comprehension with if Condition

#Program for generating even numbers upto 10

Simple for loop

```
even_numbers = []  
for x in range(10):  
    if x % 2 == 0:  
        even_numbers.append(x)
```

List comprehension

```
even_numbers = [x for x in range(10) if x % 2 == 0]  
print(even_numbers)
```

List Comprehension with if else Condition

#Program for generating even numbers upto 10 and mark odd's with - minus

Simple for loop

```
even_numbers = []  
for x in range(10):  
    if x % 2 == 0:  
        even_numbers.append(x)  
    else:  
        even_numbers.append(-x)
```

List comprehension

```
even_numbers = [x if x % 2 == 0 else -x for x in range(10)]  
print(even_numbers)
```

List Comprehension with multiple if Condition

Simple for loop

```
result = []  
for x in range(20):  
    if x % 2 == 0 and x%3==0:  
        result.append(x)
```

List comprehension

```
result= [x for x in range(20) if x % 2 == 0 and x%3==0]  
print(result)
```